

Inserting External PDF Pages

Mastering LaTeX

for Scientific Documentation



The pdfpages Package*

Andreas MATTHIAS
andreas.matthias@gmail.com

2022/12/19

Abstract

This package simplifies the insertion of external multi-page PDF or PS documents. It supports pdfTeX, VTeX, XeTeX, LuaTeX, and pTeX.

Contents

1	Introduction	1
2	Usage	2
2.1	Package Options	2
2.2	Commands	2
2.3	The Layout	11
2.4	Hints for Users of platex	11
2.5	Pitfalls	12
3	Required Packages	12
4	Acknowledgment	12

1 Introduction

When creating PDF documents, it is sometimes useful to insert pages of external PDF documents. This can be done with the `\includegraphics` command from the `graphics` package. But a simple `\includegraphics{doc.pdf}` normally produces ‘Overfull \hbox’ and ‘Overfull \vbox’ warnings, because the size of the inserted pages does not match the print space.

The `pdfpages` package makes it easy to insert pages of external PDF documents without worrying about the print space. Here are some features of the `pdfpages` package: Several logical pages can be arranged onto each sheet of paper and the layout can be changed individually. A lot of hypertext operations are supported, like links to the inserted pages, links to the original PDF document, threads, etc. When working with VTeX the same is possible with PostScript documents, too. Note that PostScript documents are only supported by VTeX and *not* by pdfLaTeX.

*This file has version number v0.5x, last revised 2022/12/19.

When producing DVI output `pdfpages` cannot insert pages of a PDF documents. But instead of interrupting execution `pdfpages` will insert empty pages. This feature is important when using packages like `pst-pdf`, which need to produce DVI output at the first run.

Links and other interactive features of PDF documents When including pages of a PDF only the so called content stream of these pages is copied but no links. Up to now there are no TeX-engines (`pdfTeX`, `XeTeX`, ...) available that can copy links or other interactive features of a PDF document, too. Thus, all kinds of links¹ will get lost during inclusion. (Using `\includepdf`, `\includegraphics`, or other low-level commands.)

However, there's a glimmer of hope. Some links may be extracted and later reinserted by a package called *pax* which can be downloaded from CTAN [3]. Have a look at it!

2 Usage

2.1 Package Options

`\usepackage[<options>]{pdfpages}`

- <option>* – **final**: Inserts pages. This is the default.
- draft**: Does not insert pages, but prints a box and the filename instead.
- demo**: Inserts empty pages instead of the actual PDFs.
- nodemo**: Disables ‘demo’.
- enable-survey**: Activates survey functionalities. (*experimental, subject to change*)

2.2 Commands

`\includepdf` Inserts pages of an external PDF document.

`\includepdf[<key=val>]{<filename>}`

- <key=val>* – A comma separated list of options using the *<key>=<value>* syntax.
- <filename>* – Filename of the PDF document. (The filename *must not* contain any blanks!)

The following list describes all possible options of `\includepdf`. All options are using the *<key=value>* syntax.

- Main options:

pages Selects pages to insert. The argument is a comma separated list, containing page numbers (`pages={3,5,6,8}`), ranges of page numbers (`pages={4-9}`) or any combination. To insert empty pages use `{}`.

¹Actually not only links but all kinds of *PDF annotations* will get lost.

the contour of this first page. This option allows to declare another page to be used as a template; which is only useful if a PDF document contains different page sizes or page orientations. The argument should be a page number. (Default: `pagetemplate=(first inserted page)`)

templatesize This option is similar to the `pagetemplate` option, but its arguments specify the size of the template directly. Its syntax is: `templatesize={width}{height}` Note: The two lengths should be a bit larger than desired, to keep away from rounding errors. (Default: `templatesize=(size of the first inserted page)`)

rotateoversize This option allows to rotate oversized pages. E.g. pages in landscape orientation are oversized relatively to their portrait counterpart, because they do not match within the contour of a portrait page without rotating them. By default oversized pages are scale and are *not* rotated. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `rotateoversize=false`)

doublepages Inserts every page twice. This is useful for 2-up printing, if one wants to cut the stack of paper afterwards to get two copies. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepages=false`)

doublepagestwist Whereas with `doublepages` the cutting edge is once on the inner side and ones on the outer side, `doublepagestwist` turns the pages such, that the cutting edge is always on the inner side. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwist=false`)

doublepagestwistodd Turns the pages such, that the cutting edge is always on the outer side. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwistodd=false`)

doublepagestwist* Like `doublepagestwist` but for double side printing. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwist*=false`)

doublepagestwistodd* Like `doublepagestwistodd` but for double side printing Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwistodd*=false`)

duplicatepages Duplicates each page n times, with n being the argument to this option. (Default: `duplicatepages=2`)

- Miscellaneous options:

lastpage In DVI mode `pdfpages` cannot determine the number of pages of the included document. So this option is suitable to specify the number of pages. This option is only used in DVI mode and has no meaning in any other mode. The argument should be a page number. (Default: `lastpage=1`)

- Hypertext options:

link Inserted pages become a target of a hyperlink. The name of the link is ‘`<filename>.<page number>`’. The filename extension of `<filename>` *must*

When producing DVI output `pdfpages` cannot insert pages of a PDF documents. But instead of interrupting execution `pdfpages` will insert empty pages. This feature is important when using packages like `pst-pdf`, which need to produce DVI output at the first run.

Links and other interactive features of PDF documents When including pages of a PDF only the so called content stream of these pages is copied but no links. Up to now there are no TeX-engines (`pdfTeX`, `XeTeX`, ...) available that can copy links or other interactive features of a PDF document, too. Thus, all kinds of links¹ will get lost during inclusion. (Using `\includepdf`, `\includegraphics`, or other low-level commands.)

However, there's a glimmer of hope. Some links may be extracted and later reinserted by a package called *pax* which can be downloaded from CTAN [3]. Have a look at it!

2 Usage

2.1 Package Options

`\usepackage[<options>]{pdfpages}`

- <option>* – **final**: Inserts pages. This is the default.
- draft**: Does not insert pages, but prints a box and the filename instead.
- demo**: Inserts empty pages instead of the actual PDFs.
- nodemo**: Disables ‘demo’.
- enable-survey**: Activates survey functionalities. (*experimental, subject to change*)

2.2 Commands

`\includepdf` Inserts pages of an external PDF document.

`\includepdf[<key=val>]{<filename>}`

- <key=val>* – A comma separated list of options using the *<key>=<value>* syntax.
- <filename>* – Filename of the PDF document. (The filename *must not* contain any blanks!)

The following list describes all possible options of `\includepdf`. All options are using the *<key=value>* syntax.

- Main options:

pages Selects pages to insert. The argument is a comma separated list, containing page numbers (`pages={3,5,6,8}`), ranges of page numbers (`pages={4-9}`) or any combination. To insert empty pages use `{}`.

¹Actually not only links but all kinds of *PDF annotations* will get lost.

E.g.: `pages={3,{},8-11,15}` will insert page 3, an empty page, and pages 8, 9, 10, 11, and 15.

Page ranges are specified by the following syntax: $\langle m \rangle - \langle n \rangle$. This selects all pages from $\langle m \rangle$ to $\langle n \rangle$. Omitting $\langle m \rangle$ defaults to the first page; omitting $\langle n \rangle$ defaults to the last page of the document. Another way to select the last page of the document, is to use the keyword `last`. (This is only permitted in a page range.)

E.g.: `pages=-` will insert *all* pages of the document, and `pages=last-1` will insert all pages in reverse order.

(Default: `pages=1`)

nup Puts multiple logical pages onto each sheet of paper. The syntax of this option is: `nup=<xnup>x<ynup>`. Where $\langle xnup \rangle$ and $\langle ynup \rangle$ specify the number of logical pages in horizontal and vertical direction, which are arranged on each sheet of paper. (Default: `nup=1x1`)

landscape Specifies the format of the sheet of paper, which is rotated by 90 degrees. This does *not* affect the logical pages, which will *not* be rotated by the ‘landscape’ option. To rotate the logical pages use the ‘angle’ option (e.g. ‘angle=90’). Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `landscape=false`)

- Layout options:

delta Puts some horizontal and vertical space between the logical pages. The argument should be two dimensions, separated by space. See Chapter 2.3 and Figure 1. (Default: `delta=0 0`).

offset Displaces the origin of the inserted pages. The argument should be two dimensions, separated by space. In ‘oneside’ documents positive values shift the pages to the *right* and to the *top* margin, respectively, whereas in ‘twoside’ documents positive values shift the pages to the *outer* and to the *top* margin, respectively. See Chapter 2.3 and Figure 1. (Default: `offset=0 0`)

frame Puts a frame around each logical page. The frame is made of lines of thickness `\fboxrule`. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `frame=false`)

column Pdfpages normally uses ‘row-major’ layout, where successive pages are placed in rows along the paper. The `column` option changes the output into a ‘column-major’ layout, where successive pages are arranged in columns down the paper. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `column=false`)

columnstrict By default the last page is not set in a strict ‘column-major’ layout, if the logical pages do not fill up the whole page. The `columnstrict` option forces a strict ‘column-major’ layout for the last page. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `columnstrict=false`)

1	4	
2	5	
3		

`columnstrict=true`

1	3	5
2	4	

`columnstrict=false`

openright This option puts an empty page before the first logical page. In combination with `nup=2x1`, `nup=2x2`, etc., this means that the first page is on the right side. The same effect can be achieved with the `pages` option, if an empty page is inserted in front of the first page. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `openright=false`)

openrighteach Similar to `openright` but affects *each* file processed by `\includepdfmerge`.

pagecommand Declares L^AT_EX commands, which are executed on each sheet of paper. (Default: `pagecommand={\thispagestyle{empty}}`)

turn By default pages in landscape format are displayed in landscape orientation (if the PDF viewer supports this). With `turn=false` this can be prohibited. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `turn=true`)

noautoscale By default pages are scaled automatically. This can be suppressed with the `noautoscale` option. In combination with the `scale` option (from `graphicx`) the user has full control over the scaling process. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `noautoscale=false`)

fitpaper Adjusts the paper size to the one of the inserted document. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `fitpaper=false`)

reflect Reflects included pages. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `reflect=false`)

signature Creates booklets by rearranging pages into signatures and setting `nup=1x2` or `nup=2x1`, respectively. This option takes one argument specifying the size of the signature, which should be a multiple of 4.
An example for documents in portrait orientation:
`\includepdf[pages=-, signature=8, landscape]{portrait-doc.pdf}`
An example for documents in landscape orientation:
`\includepdf[pages=-, signature=8]{landscape-doc.pdf}`

signature* Similar to `signature`, but now for right-edge binding.

booklet This option is just a shortcut of the ‘signature’ option, if you choose a signature value so large that all pages fit into one signature. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `booklet=false`)

picturecommand Declares picture commands which are executed on every page within a picture environment with the base point at the lower left corner of the page. (The base point does not change if the page is rotated, e.g. by the `landscape` option.) (Default: `picturecommand={}`)

picturecommand* Like `picturecommand`, but with the restriction that `picturecommand*` executes its picture commands only on the very first page. (Default: `picturecommand*={}`)

pagetemplate By default the first inserted page will be used as a template. This means that all further pages are scaled such that they match within

the contour of this first page. This option allows to declare another page to be used as a template; which is only useful if a PDF document contains different page sizes or page orientations. The argument should be a page number. (Default: `pagetemplate=(first inserted page)`)

templatesize This option is similar to the `pagetemplate` option, but its arguments specify the size of the template directly. Its syntax is: `templatesize={width}{height}` Note: The two lengths should be a bit larger than desired, to keep away from rounding errors. (Default: `templatesize=(size of the first inserted page)`)

rotateoversize This option allows to rotate oversized pages. E.g. pages in landscape orientation are oversized relatively to their portrait counterpart, because they do not match within the contour of a portrait page without rotating them. By default oversized pages are scale and are *not* rotated. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `rotateoversize=false`)

doublepages Inserts every page twice. This is useful for 2-up printing, if one wants to cut the stack of paper afterwards to get two copies. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepages=false`)

doublepagestwist Whereas with `doublepages` the cutting edge is once on the inner side and ones on the outer side, `doublepagestwist` turns the pages such, that the cutting edge is always on the inner side. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwist=false`)

doublepagestwistodd Turns the pages such, that the cutting edge is always on the outer side. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwistodd=false`)

doublepagestwist* Like `doublepagestwist` but for double side printing. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwist*=false`)

doublepagestwistodd* Like `doublepagestwistodd` but for double side printing Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwistodd*=false`)

duplicatepages Duplicates each page n times, with n being the argument to this option. (Default: `duplicatepages=2`)

- Miscellaneous options:

lastpage In DVI mode `pdfpages` cannot determine the number of pages of the included document. So this option is suitable to specify the number of pages. This option is only used in DVI mode and has no meaning in any other mode. The argument should be a page number. (Default: `lastpage=1`)

- Hypertext options:

link Inserted pages become a target of a hyperlink. The name of the link is ‘`<filename>.<page number>`’. The filename extension of `<filename>` must

not be stripped. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `link=false`)

linkname Changes the default linkname created by the option `link`. Instead of `<filename>` the value of this option is used. E.g. `linkname=mylink` produces the linknames ‘`mylink.<page number>`’.

thread Combines inserted pages to an article thread. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `thread=false`)

threadname Several threads are distinguished by their threadnames. By default the threadname is equal to the filename (plus filename extension), but it can be changed with this option. This is useful if the same file is inserted twice or more times and should not be combined to one single thread. Or the other way round if pages from different documents should be combined to one single thread. (Default: `threadname=<filename.ext>`)

linktodoc Lets the inserted pages be hyperlinks to the document from which they were extracted. Note that the PDF-Viewer will not find the file, if filename extension (.pdf) is missing in `<filename>`. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `linktodoc=false`)

- Additional hypertext options:

linkfit Specifies, how the viewer displays a linked page. This option changes the default behavior of the option `link`. Possible values are: `Fit`, `FitH <top>`, `FitV <left>`, `FitB`, `FitBH <top>`, `FitBV <left>`, and `Region`.

See [2] for a details description of these PDF destinations. The `region` destination was added by `pdfpages` and is not a real PDF destinations. It scales a page such that the included page fits exactly into the window of the PDF viewer.

Note that not all of these options are supported by all T_EX-engines or drivers, respectively. (Default: `linkfit=fit`)

linktodocfit By default the option `linktodoc` opens the page in ‘Fit in Window’ view. Another view can be specified with this option. Possible values are the legal PDF tokens: `/FitH <top>`, `/FitV <left>`, etc. (See [2] for more details.) (Default: `linktodocfit=/Fit`)

newwindow By default option `linktodoc` opens a new window. This can be changed with option `newwindow`. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `newwindow=true`)

linkfilename Sets the name (with path) of the file to be linked to by the option `linktodoc`. You will hardly ever need this option. (Default: `linkfilename=<filename.ext>`)

- Experimental options: (Syntax may change in future versions!)

addtotoc Adds an entry to the table of contents. This option requires five arguments, separated by commas:

`addtotoc={<page number>,<section>,<level>,<heading>,<label>}`

`<page number>`: Page number of the inserted page.

The pdfpages Package*

Andreas MATTHIAS
andreas.matthias@gmail.com

2022/12/19

Abstract

This package simplifies the insertion of external multi-page PDF or PS documents. It supports pdfTeX, VTeX, XeTeX, LuaTeX, and pTeX.

Contents

1	Introduction	1
2	Usage	2
2.1	Package Options	2
2.2	Commands	2
2.3	The Layout	11
2.4	Hints for Users of platex	11
2.5	Pitfalls	12
3	Required Packages	12
4	Acknowledgment	12

1 Introduction

When creating PDF documents, it is sometimes useful to insert pages of external PDF documents. This can be done with the `\includegraphics` command from the `graphics` package. But a simple `\includegraphics{doc.pdf}` normally produces ‘Overfull \hbox’ and ‘Overfull \vbox’ warnings, because the size of the inserted pages does not match the print space.

The `pdfpages` package makes it easy to insert pages of external PDF documents without worrying about the print space. Here are some features of the `pdfpages` package: Several logical pages can be arranged onto each sheet of paper and the layout can be changed individually. A lot of hypertext operations are supported, like links to the inserted pages, links to the original PDF document, threads, etc. When working with VTeX the same is possible with PostScript documents, too. Note that PostScript documents are only supported by VTeX and *not* by pdfLaTeX.

*This file has version number v0.5x, last revised 2022/12/19.

E.g.: `pages={3,{},8-11,15}` will insert page 3, an empty page, and pages 8, 9, 10, 11, and 15.

Page ranges are specified by the following syntax: $\langle m \rangle - \langle n \rangle$. This selects all pages from $\langle m \rangle$ to $\langle n \rangle$. Omitting $\langle m \rangle$ defaults to the first page; omitting $\langle n \rangle$ defaults to the last page of the document. Another way to select the last page of the document, is to use the keyword `last`. (This is only permitted in a page range.)

E.g.: `pages=-` will insert *all* pages of the document, and `pages=last-1` will insert all pages in reverse order.

(Default: `pages=1`)

nup Puts multiple logical pages onto each sheet of paper. The syntax of this option is: `nup=<xnup>x<ynup>`. Where $\langle xnup \rangle$ and $\langle ynup \rangle$ specify the number of logical pages in horizontal and vertical direction, which are arranged on each sheet of paper. (Default: `nup=1x1`)

landscape Specifies the format of the sheet of paper, which is rotated by 90 degrees. This does *not* affect the logical pages, which will *not* be rotated by the ‘landscape’ option. To rotate the logical pages use the ‘angle’ option (e.g. ‘angle=90’). Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `landscape=false`)

- Layout options:

delta Puts some horizontal and vertical space between the logical pages. The argument should be two dimensions, separated by space. See Chapter 2.3 and Figure 1. (Default: `delta=0 0`).

offset Displaces the origin of the inserted pages. The argument should be two dimensions, separated by space. In ‘oneside’ documents positive values shift the pages to the *right* and to the *top* margin, respectively, whereas in ‘twoside’ documents positive values shift the pages to the *outer* and to the *top* margin, respectively. See Chapter 2.3 and Figure 1. (Default: `offset=0 0`)

frame Puts a frame around each logical page. The frame is made of lines of thickness `\fbboxrule`. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `frame=false`)

column Pdfpages normally uses ‘row-major’ layout, where successive pages are placed in rows along the paper. The `column` option changes the output into a ‘column-major’ layout, where successive pages are arranged in columns down the paper. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `column=false`)

columnstrict By default the last page is not set in a strict ‘column-major’ layout, if the logical pages do not fill up the whole page. The `columnstrict` option forces a strict ‘column-major’ layout for the last page. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `columnstrict=false`)

1	4	
2	5	
3		

`columnstrict=true`

1	3	5
2	4	

`columnstrict=false`

openright This option puts an empty page before the first logical page. In combination with `nup=2x1`, `nup=2x2`, etc., this means that the first page is on the right side. The same effect can be achieved with the `pages` option, if an empty page is inserted in front of the first page. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `openright=false`)

openrighteach Similar to `openright` but affects *each* file processed by `\includepdfmerge`.

pagecommand Declares L^AT_EX commands, which are executed on each sheet of paper. (Default: `pagecommand={\thispagestyle{empty}}`)

turn By default pages in landscape format are displayed in landscape orientation (if the PDF viewer supports this). With `turn=false` this can be prohibited. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `turn=true`)

noautoscale By default pages are scaled automatically. This can be suppressed with the `noautoscale` option. In combination with the `scale` option (from `graphicx`) the user has full control over the scaling process. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `noautoscale=false`)

fitpaper Adjusts the paper size to the one of the inserted document. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `fitpaper=false`)

reflect Reflects included pages. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `reflect=false`)

signature Creates booklets by rearranging pages into signatures and setting `nup=1x2` or `nup=2x1`, respectively. This option takes one argument specifying the size of the signature, which should be a multiple of 4.
An example for documents in portrait orientation:
`\includepdf[pages=-, signature=8, landscape]{portrait-doc.pdf}`
An example for documents in landscape orientation:
`\includepdf[pages=-, signature=8]{landscape-doc.pdf}`

signature* Similar to `signature`, but now for right-edge binding.

booklet This option is just a shortcut of the ‘signature’ option, if you choose a signature value so large that all pages fit into one signature. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `booklet=false`)

picturecommand Declares picture commands which are executed on every page within a picture environment with the base point at the lower left corner of the page. (The base point does not change if the page is rotated, e.g. by the `landscape` option.) (Default: `picturecommand={}`)

picturecommand* Like `picturecommand`, but with the restriction that `picturecommand*` executes its picture commands only on the very first page. (Default: `picturecommand*={}`)

pagetemplate By default the first inserted page will be used as a template. This means that all further pages are scaled such that they match within

the contour of this first page. This option allows to declare another page to be used as a template; which is only useful if a PDF document contains different page sizes or page orientations. The argument should be a page number. (Default: `pagetemplate=(first inserted page)`)

templatesize This option is similar to the `pagetemplate` option, but its arguments specify the size of the template directly. Its syntax is: `templatesize={width}{height}` Note: The two lengths should be a bit larger than desired, to keep away from rounding errors. (Default: `templatesize=(size of the first inserted page)`)

rotateoversize This option allows to rotate oversized pages. E.g. pages in landscape orientation are oversized relatively to their portrait counterpart, because they do not match within the contour of a portrait page without rotating them. By default oversized pages are scale and are *not* rotated. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `rotateoversize=false`)

doublepages Inserts every page twice. This is useful for 2-up printing, if one wants to cut the stack of paper afterwards to get two copies. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepages=false`)

doublepagestwist Whereas with `doublepages` the cutting edge is once on the inner side and ones on the outer side, `doublepagestwist` turns the pages such, that the cutting edge is always on the inner side. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwist=false`)

doublepagestwistodd Turns the pages such, that the cutting edge is always on the outer side. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwistodd=false`)

doublepagestwist* Like `doublepagestwist` but for double side printing. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwist*=false`)

doublepagestwistodd* Like `doublepagestwistodd` but for double side printing Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `doublepagestwistodd*=false`)

duplicatepages Duplicates each page n times, with n being the argument to this option. (Default: `duplicatepages=2`)

- Miscellaneous options:

lastpage In DVI mode `pdfpages` cannot determine the number of pages of the included document. So this option is suitable to specify the number of pages. This option is only used in DVI mode and has no meaning in any other mode. The argument should be a page number. (Default: `lastpage=1`)

- Hypertext options:

link Inserted pages become a target of a hyperlink. The name of the link is ‘`<filename>.<page number>`’. The filename extension of `<filename>` must

not be stripped. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `link=false`)

linkname Changes the default linkname created by the option `link`. Instead of $\langle filename \rangle$ the value of this option is used. E.g. `linkname=mylink` produces the linknames ‘mylink. $\langle page\ number \rangle$ ’.

thread Combines inserted pages to an article thread. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `thread=false`)

threadname Several threads are distinguished by their threadnames. By default the threadname is equal to the filename (plus filename extension), but it can be changed with this option. This is useful if the same file is inserted twice or more times and should not be combined to one single thread. Or the other way round if pages from different documents should be combined to one single thread. (Default: `threadname=\langle filename.ext \rangle`)

linktodoc Lets the inserted pages be hyperlinks to the document from which they were extracted. Note that the PDF-Viewer will not find the file, if filename extension (.pdf) is missing in $\langle filename \rangle$. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `linktodoc=false`)

- Additional hypertext options:

linkfit Specifies, how the viewer displays a linked page. This option changes the default behavior of the option `link`. Possible values are: `Fit`, `FitH` $\langle top \rangle$, `FitV` $\langle left \rangle$, `FitB`, `FitBH` $\langle top \rangle$, `FitBV` $\langle left \rangle$, and `Region`.

See [2] for a details description of these PDF destinations. The `region` destination was added by pdfpages and is not a real PDF destinations. It scales a page such that the included page fits exactly into the window of the PDF viewer.

Note that not all of these options are supported by all T_EX-engines or drivers, respectively. (Default: `linkfit=fit`)

linktodocfit By default the option `linktodoc` opens the page in ‘Fit in Window’ view. Another view can be specified with this option. Possible values are the legal PDF tokens: `/FitH` $\langle top \rangle$, `/FitV` $\langle left \rangle$, etc. (See [2] for more details.) (Default: `linktodocfit=/Fit`)

newwindow By default option `linktodoc` opens a new window. This can be changed with option `newwindow`. Either ‘true’ or ‘false’ (or no value, which is equivalent to ‘true’). (Default: `newwindow=true`)

linkfilename Sets the name (with path) of the file to be linked to by the option `linktodoc`. You will hardly ever need this option. (Default: `linkfilename=\langle filename.ext \rangle`)

- Experimental options: (Syntax may change in future versions!)

addtotoc Adds an entry to the table of contents. This option requires five arguments, separated by commas:

`addtotoc=\{ \langle page\ number \rangle, \langle section \rangle, \langle level \rangle, \langle heading \rangle, \langle label \rangle \}`

$\langle page\ number \rangle$: Page number of the inserted page.

$\langle section \rangle$: L^AT_EX sectioning name – e.g., section, subsection, ...

$\langle level \rangle$: Number, denoting depth of section – e.g., 1 for section level, 2 for subsection level, ...

$\langle heading \rangle$: Title inserted in the table of contents.

$\langle label \rangle$: Name of the label. This label can be referred to with `\ref` and `\pageref`.

Note: The order of the five arguments must not be mixed. Otherwise you will get very strange error messages.

The `addtotoc` option accepts multiple sets of the above mentioned five arguments, all separated by commas. The sets must be sorted such that the $\langle page\ number \rangle$ s are in ascending order. (Strictly speaking they must have the same order as the page numbers specified by the `pages` option.)

The proper recursive definition of the `addtotoc` option is:

```
addtotoc={\toc-list}
```

```
\toc-list → \page number,\section,\level,\heading,\label[\toc-list]
```

addtolist Adds an entry to the list of figures, the list of tables, or any other list (e.g. from *float.sty*). This option requires four arguments, separated by commas:

```
addtolist={\page number,\type,\heading,\label}
```

$\langle page\ number \rangle$: Page number of the inserted page.

$\langle type \rangle$: Name of a floating environment. (`figure`, `table`, etc.)

$\langle heading \rangle$: Title inserted into LoF, LoT, etc.

$\langle label \rangle$: Name of the label. This label can be referred to with `\ref` and `\pageref`.

Like `addtotoc`, `addtolist` accepts multiple sets of the above mentioned four arguments, all separated by commas. The proper recursive definition is:

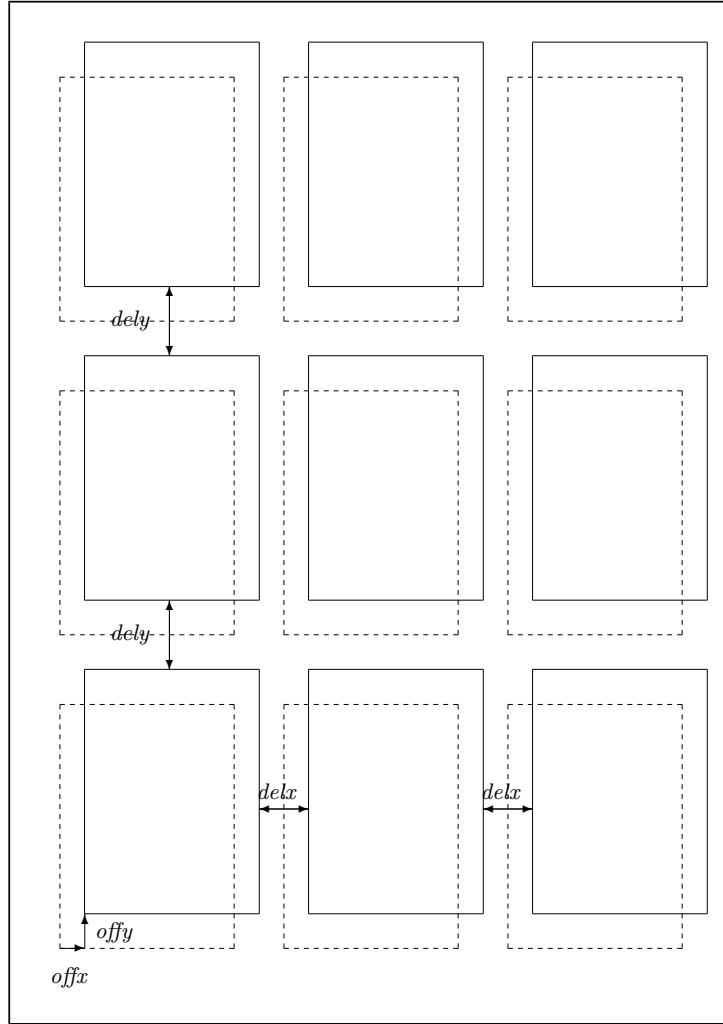
```
addtolist={\lof-list}
```

```
\lof-list → \page number,\type,\heading,\label[\lof-list]
```

survey Creates a survey of those pages of the document, which are marked with `\AddToSurvey`. (`\AddToSurvey` is a simple command with no arguments. It just writes out labels to the `.aux` file.) This option may be used when preparing slides to create a survey of only ‘finished’ pages – if pages are build up incrementally.

To use this option a special sequence of production steps must be obeyed. Here is a small example:

```
--- slides.tex ---
\documentclass{article}
\usepackage[draft,enable-survey]{pdfpages}
\begin{document}
  ... some text ...
  \AddToSurvey
  ... some text ...
  \includepdf[survey,nup=2x2]{slides-tmp.pdf}
\end{document}
```

— physical sheet of paper	$\mathbf{delta}=\langle delx \rangle \langle dely \rangle$
— logical pages	$\mathbf{offset}=\langle offx \rangle \langle offy \rangle$
--- centered position	

Figure 1: Layout

`\includepdfmerge` Inserts pages of several external PDF documents.

```
\includepdfmerge[ $\langle key=val \rangle$ ]{ $\langle file-page-list \rangle$ }
```

$\langle key=val \rangle$ – A comma separated list of options using the $\langle key \rangle=\langle value \rangle$ syntax.
 $\langle file-page-list \rangle$ – $\langle filename \rangle$ [, $\langle page spec \rangle$][, $\langle file-page-list \rangle$]
A comma separated list of filenames and optional $\langle page spec \rangle$ specifiers. A $\langle page spec \rangle$ can be everything the option `pages` accepts.

The `\includepdfmerge` command uses the same options as `\includepdf` with one exception. The option `pages` has no meaning for `\includepdfmerge`. Instead the $\langle page spec \rangle$ specifier is used to specify which pages should be inserted. The $\langle page spec \rangle$ specifier accepts the same values as the `pages` option. If no $\langle page spec \rangle$ specifier is given, only the first page will be inserted.

Examples: To create a kind of summary of three PDF documents, it might be nice to insert just the first page of each document and to provide links to the original documents:

```
\includepdfmerge[nup=1x3, landscape, linktodoc]  
{doc1.pdf, doc2.pdf, doc3.pdf}
```

But sometimes the title page of a document is not the first page. So it would be more pleasant to insert the title page of each document than the first page. This can be done with the $\langle page spec \rangle$ specifier. The following example inserts the second page of *doc1.pdf* and the third page of *doc2.pdf* and *doc3.pdf*:

```
\includepdfmerge[nup=1x3, landscape, linktodoc]  
{doc1.pdf, 2, doc2.pdf, 3, doc3.pdf, 3}
```

Here is an example of more complex $\langle page spec \rangle$ specifiers:

```
\includepdfmerge[nup=1x3, landscape, linktodoc]  
{doc1.pdf, 1-3,  
 doc2.pdf, 3, 5, 9,  
 doc3.pdf, 3-5, 7}
```

`\includepdfset` If you need the same options for `\includepdf` all the time, it is possible to define global options with `\includepdfset`. The argument of `\includepdfset` is a comma separated list of options, using the $\langle key \rangle=\langle value \rangle$ syntax. These options are processed each time `\includepdf` is called. Local options (passed as an optional argument directly to `\includepdf`) are overwriting global options:

```
\includepdfset{ $\langle global options \rangle$ }  
\includepdf[ $\langle local options \rangle$ ]{pdf-file}
```

The pdfpages Package*

Andreas MATTHIAS
andreas.matthias@gmail.com

2022/12/19

Abstract

This package simplifies the insertion of external multi-page PDF or PS documents. It supports pdfTeX, VTeX, XeTeX, LuaTeX, and pTeX.

Contents

1	Introduction	1
2	Usage	2
2.1	Package Options	2
2.2	Commands	2
2.3	The Layout	11
2.4	Hints for Users of platex	11
2.5	Pitfalls	12
3	Required Packages	12
4	Acknowledgment	12

1 Introduction

When creating PDF documents, it is sometimes useful to insert pages of external PDF documents. This can be done with the `\includegraphics` command from the `graphics` package. But a simple `\includegraphics{doc.pdf}` normally produces ‘`Overfull \hbox`’ and ‘`Overfull \vbox`’ warnings, because the size of the inserted pages does not match the print space.

The `pdfpages` package makes it easy to insert pages of external PDF documents without worrying about the print space. Here are some features of the `pdfpages` package: Several logical pages can be arranged onto each sheet of paper and the layout can be changed individually. A lot of hypertext operations are supported, like links to the inserted pages, links to the original PDF document, threads, etc. When working with VTeX the same is possible with PostScript documents, too. Note that PostScript documents are only supported by VTeX and *not* by pdfLaTeX.

*This file has version number v0.5x, last revised 2022/12/19.

When producing DVI output `pdfpages` cannot insert pages of a PDF documents. But instead of interrupting execution `pdfpages` will insert empty pages. This feature is important when using packages like `pst-pdf`, which need to produce DVI output at the first run.

Links and other interactive features of PDF documents When including pages of a PDF only the so called content stream of these pages is copied but no links. Up to now there are no TeX-engines (pdfTeX, XeTeX, ...) available that can copy links or other interactive features of a PDF document, too. Thus, all kinds of links¹ will get lost during inclusion. (Using `\includepdf`, `\includegraphics`, or other low-level commands.)

However, there's a glimmer of hope. Some links may be extracted and later reinserted by a package called `pax` which can be downloaded from CTAN [3]. Have a look at it!

2 Usage

2.1 Package Options

`\usepackage[(options)]{pdfpages}`

(*option*) – **final**: Inserts pages. This is the default.

draft: Does not insert pages, but prints a box and the filename instead.

demo: Inserts empty pages instead of the actual PDFs.

nodemo: Disables ‘demo’.

enable-survey: Activates survey functionalities. (*experimental, subject to change*)

2.2 Commands

`\includepdf` Inserts pages of an external PDF document.

`\includepdf[(key=val)]{filename}`

(*key=val*) – A comma separated list of options using the

(*key*)=(*value*) syntax.

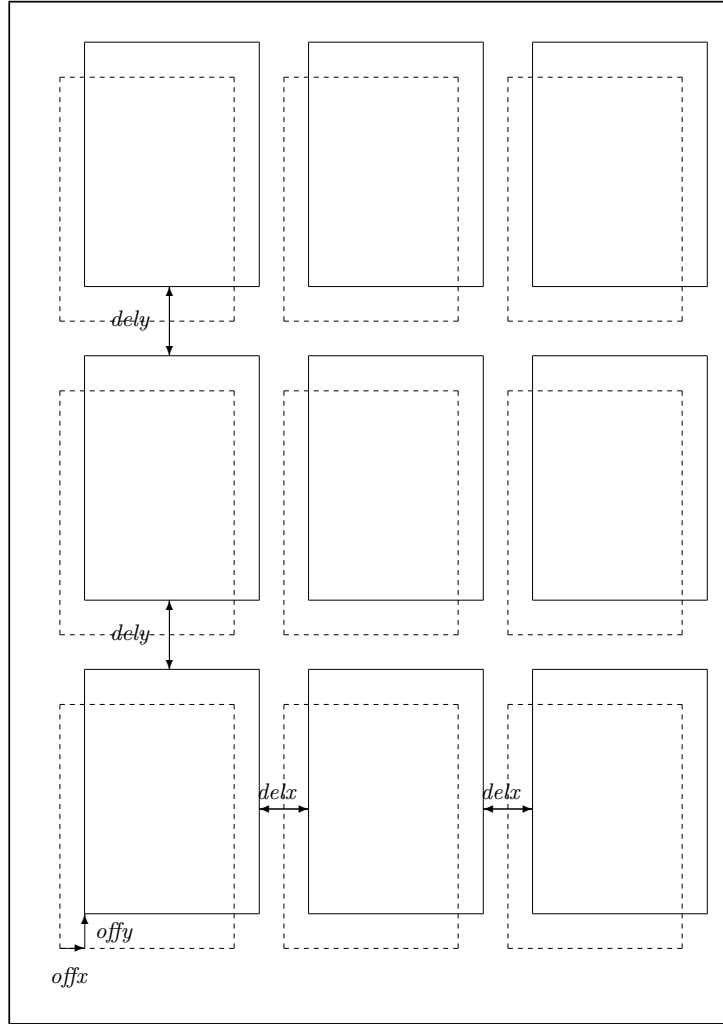
(*filename*) – Filename of the PDF document. (The filename *must* not contain any blanks!)

The following list describes all possible options of `\includepdf`. All options are using the (*key=value*) syntax.

- Main options:

pages Selects pages to insert. The argument is a comma separated list, containing page numbers (`pages={3,5,6,8}`), ranges of page numbers (`pages={4-9}`) or any combination. To insert empty pages use {}.

¹Actually not only links but all kinds of PDF annotations will get lost.



— physical sheet of paper	$\mathbf{delta}=\langle delx \rangle \langle dely \rangle$
— logical pages	$\mathbf{offset}=\langle offx \rangle \langle offy \rangle$
--- centered position	

Figure 1: Layout

The pdfpages Package*

Andreas MATTHIAS
andreas.matthias@gmail.com

2022/12/19

Abstract

This package simplifies the insertion of external multi-page PDF or PS documents. It supports pdfTeX, VTeX, XeTeX, LuaTeX, and pTeX.

Contents

1 Introduction	1
2 Usage	2
2.1 Package Options	2
2.2 Commands	2
2.3 The Layout	11
2.4 Hints for Users of platex	11
2.5 Pitfalls	12
3 Required Packages	12
4 Acknowledgment	12

1 Introduction

When creating PDF documents, it is sometimes useful to insert pages of external PDF documents. This can be done with the `\includegraphics` command from the `graphics` package. But a simple `\includegraphics{doc.pdf}` normally produces ‘`Overfull \hbox`’ and ‘`Overfull \vbox`’ warnings, because the size of the inserted pages does not match the print space.

The `pdfpages` package makes it easy to insert pages of external PDF documents without worrying about the print space. Here are some features of the `pdfpages` package: Several logical pages can be arranged onto each sheet of paper and the layout can be changed individually. A lot of hypertext operations are supported, like links to the inserted pages, links to the original PDF document, threads, etc. When working with VTeX the same is possible with PostScript documents, too. Note that PostScript documents are only supported by VTeX and *not* by pdfLaTeX.

*This file has version number v0.5x, last revised 2022/12/19.

When producing DVI output `pdfpages` cannot insert pages of a PDF documents. But instead of interrupting execution `pdfpages` will insert empty pages. This feature is important when using packages like `pst-pdf`, which need to produce DVI output at the first run.

Links and other interactive features of PDF documents When including pages of a PDF only the so called content stream of these pages is copied but no links. Up to now there are no TeX-engines (pdfTeX, XeTeX, ...) available that can copy links or other interactive features of a PDF document, too. Thus, all kinds of links¹ will get lost during inclusion. (Using `\includepdf`, `\includegraphics`, or other low-level commands.)

However, there’s a glimmer of hope. Some links may be extracted and later reinserted by a package called `pax` which can be downloaded from CTAN [3]. Have a look at it!

2 Usage

2.1 Package Options

`\usepackage[options]{pdfpages}`

(option) – **final**: Inserts pages. This is the default.

draft: Does not insert pages, but prints a box and the filename instead.

demo: Inserts empty pages instead of the actual PDFs.

nodemo: Disables ‘demo’.

enable-survey: Activates survey functionalities. (*experimental, subject to change*)

2.2 Commands

`\includepdf` Inserts pages of an external PDF document.

`\includepdf[(key=val)]{(filename)}`

(key=val) – A comma separated list of options using the *(key)=(value)* syntax.

(filename) – Filename of the PDF document. (The filename *must* not contain any blanks!)

The following list describes all possible options of `\includepdf`. All options are using the *(key=value)* syntax.

• Main options:

pages Selects pages to insert. The argument is a comma separated list, containing page numbers (`pages={3,5,6,8}`), ranges of page numbers (`pages={4-9}`) or any combination. To insert empty pages use `{}`.

¹Actually not only links but all kinds of PDF annotations will get lost.

[illegible]

Mastering LaTeX

for Scientific Documentation



The pdfpages Package*

Andreas MATHIAS
andreas.mathias@gmail.com
2022/12/19

Abstract

This package simplifies the insertion of external multi-page PDF or PS documents. It supports pdfTeX, XeTeX, LaTeX, LuaTeX, and pdfLaTeX.

Contents

1	Introduction	1
2	Usage	2
2.1	Package Options	2
2.2	Commands	2
2.3	The Layout	11
2.4	How to Use of photo	11
2.5	Printable	12
3	Required Packages	12
4	Acknowledgment	12

1 Introduction

When creating PDF documents, it is sometimes useful to insert pages of external PDF documents. This can be done with the `\includegraphics` command from the `graphics` package. But a simple `\includegraphics{file.pdf}` normally produces `Standard: Error! (file not found)`. When working with XeTeX, the error message becomes the size of the inserted page does not match the print space.

The `pdfpages` package enables a user to insert pages of external PDF documents without worrying about the print space. Here are some features of the `pdfpages` package: Several input pages can be arranged onto each sheet of paper and the layout can be changed individually. A lot of separate operations are supported, like links to the inserted pages, links to the original PDF document, `showthis`, etc. When working with XeTeX, the same is possible with `pdfpages` documents, too. Note that `pdfpages` documents are only supported by XeTeX and not by pdfLaTeX.

*This file has version number 0.25, last revised 2022/12/19.

When producing DVI output `pdfpages` cannot insert pages of a PDF document, but instead of inserting external `pdfpages` will insert empty pages. This feature is important when using packages like `ps-gs`, which need to produce DVI output at the first run.

Links and other interactive features of PDF documents. When including pages of a PDF only the so-called content streams of those pages is copied but no links. Up to now there are no TeX engines (pdfTeX, XeTeX, ...) available that can copy links or other interactive features of a PDF document, too. Thus, all links of links will get lost during inclusion. (Using `\includepdf`, `\includegraphics`, or other `key=value` commands.)

However, there is a feature of `pdfpages`. When links may be extracted and later inserted by a package called `pdf` which can be downloaded from CTAN [2]. Here is a link to it!

2 Usage

2.1 Package Options

`\usepackage[options]{pdfpages}`

- `(option)` - `final`: Insert pages. This is the default.
- `(option)` - `draft`: Does not insert pages, but prints a box and the filename instead.
- `(option)` - `demo`: Inserts empty pages instead of the actual PDFs.
- `(option)` - `noembed`: Disables `showthis`.
- `(option)` - `enable-survey`: Activates survey functionalities. (experimental, subject to change)

2.2 Commands

`\includepdf` Inserts pages of an external PDF document.

`\includepdf[options]{filenames}`

- `(option)` - `fit`: A comma separated list of options using the `(key=value)` syntax.
- `(option)` - `fitmax`: Fitmax of the PDF document. (The filename must not contain any blanks!)

The following list describes all possible options of `\includepdf`. All options are using the `(key=value)` syntax.

• Main options:

pages Selects pages to insert. The argument is a comma separated list, containing page numbers (`pages={1,2,3,4,5}`), ranges of page numbers (`pages={6-8}`) or any combination. It inserts empty pages on 0.

*Actually not only links but all kinds of PDF annotations will get lost.

Mastering LaTeX

for Scientific Documentation

